MP 0490 · Programación de servicios y procesos
AEA1 · RA1 · Curso 2026-2027

Práctica 01
Supervisor de procesos y comunicación IPC

Informe, evidencias y decisiones

Alumno/a: Vlad Bobu

Repositorio: https://github.com/VladIESEBRE/Practica01-Supervisor-IPC

Fecha: octubre de 2026

Tecnología: Java 21, Maven, JUnit 5, Linux (systemd)

1. Resumen

La aplicación es un **supervisor** escrito en Java que crea procesos reales (cada uno con su PID), les envía datos por stdin, lee su stdout y su stderr a la vez, controla el tiempo máximo de cada uno y los para sin dejar procesos huérfanos. También conecta un productor con un consumidor mediante una pipe implementada por el propio programa.

Todo está en un único programa (`supervisor.jar`, clase `practica01.ProcessForkJoinDemo`), que hace un rol u otro según el primer argumento. El supervisor se lanza a sí mismo con otro rol para crear cada proceso hijo.

Argumentos	**Rol**	**Qué hace**
(ninguno)	SUPERVISOR	Lanza y vigila a los demás procesos y agrega los resultados
worker <NOMBRE> <pasos> <código>	WORKER	Hace <pasos> pasos de 1 s y termina con <código>
producer	PRODUCER	Escribe cinco nombres por stdout
consumer	CONSUMER	Lee stdin hasta EOF y responde COUNT=<líneas> CHARS=<caracteres>

Todos los mensajes de diagnóstico identifican rol, PID, PPID e hilo:

```
[WORKER-BETA pid=54035 ppid=54010 thread=main] pas 2/10
```

Escenario de la demo: tres workers (ALPHA acaba bien, BETA supera el tiempo límite de 5 s y GAMMA acaba con el código 7) con un máximo de dos a la vez, y después la pipe producer → supervisor → consumer.

2. Diseño y decisiones

2.1 Clases

Clase	**Responsabilidad**
ProcessForkJoinDemo	Punto de entrada: lee el rol del primer argumento y termina con System.exit(código)
Log	Mensajes con el formato [ROL pid ppid thread]; info() por stdout, error() y diagnostico() por stderr
Worker / Producer / Consumer	Los roles de los procesos hijos
Supervisor	Lanza los hijos (ProcessBuilder), envía stdin, espera con timeout, los para (ordenada y forzada), pool de 2 hilos, pipe y shutdown hook
LectorFlujo	Hilo que lee un tubo de salida de un hijo línea a línea, con hora y origen; opcionalmente reenvía cada línea al stdin de otro proceso (la pipe)
Resultado	record inmutable: orden, PID, PPID, código, timeout, duración, stdout y stderr

2.2 Decisiones

- **Un solo programa con roles** en lugar de órdenes del sistema o de cuatro programas: una sola compilación, el mismo formato de log para todos, control total del comportamiento de los hijos (duración, salidas, código) y pruebas reproducibles que solo necesitan Java.
- **Argumentos como lista** en ProcessBuilder (nunca `sh -c` con un texto): ninguna shell interpreta caracteres especiales, así que no hay inyección de órdenes.
- **Un hilo lector por tubo** (io-stdout, io-stderr), arrancados justo después de start(): ningún búfer se llena y se evita el interbloqueo.

- **Cierre de stdin** después de escribir (try-with-resources): el hijo recibe EOF y no se queda esperando.
- **Parada en dos fases**: destroy() (SIGTERM) y, si al cabo de 2 s de gracia sigue vivo, destroyForcibly() (SIGKILL), también para los descendientes.
- **Shutdown hook** + conjunto ACTIVOS + indicador `cerrando`: si el supervisor recibe Ctrl+C o kill, para a todos sus hijos y no lanza ninguno nuevo.
- **Pool de 2 hilos** (Executors.newFixedThreadPool(2)): cada hilo lanza un worker y espera a que acabe, de modo que nunca hay más de dos workers vivos a la vez.
- **Resultado inmutable** (record con copias List.copyOf): se puede pasar entre hilos sin riesgo.
- **Código de salida del supervisor**: 0 si todos los hijos acaban bien, 1 si alguno falla (código distinto de 0 o timeout). Así el error es explícito también para la shell y para systemd.

2.3 Flujo IPC

```
PRODUCER --stdout--> SUPERVISOR (hilo "pipe") --stdin--> CONSUMER
                            |
                            '--> resultado y diagnosis (stdout/stderr del supervisor)
```

3. Desarrollo y evidencias

3.1 Identificación de procesos e hilos

Primera observación del supervisor con `ps`, `pstree` y `ps -T`. Un proceso Java tiene muchos hilos (los de la JVM) y todos comparten el PID; `ps -T` muestra un SPID (identificador de hilo) diferente para cada uno. El estado `Sl+` significa: dormido (S), multihilo (l) y en primer plano (+). Ctrl+C devuelve el código 130 = 128 + 2 (SIGINT).

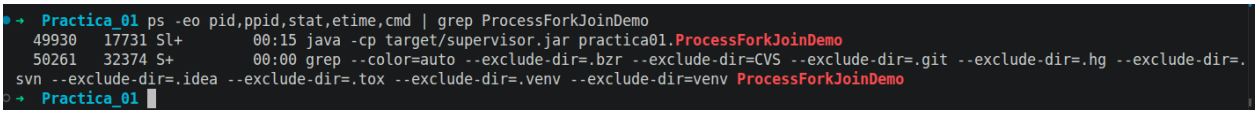


Figura 1. ps: el supervisor (PPID = zsh) y la línea del propio grep



Figura 2. Código 130 después de Ctrl+C (SIGINT)

3.2 Lanzar un proceso con ProcessBuilder

El supervisor ejecuta el mismo programa con el rol worker. El hijo tiene un PID nuevo y su PPID es el supervisor; waitFor() devuelve su código de retorno.

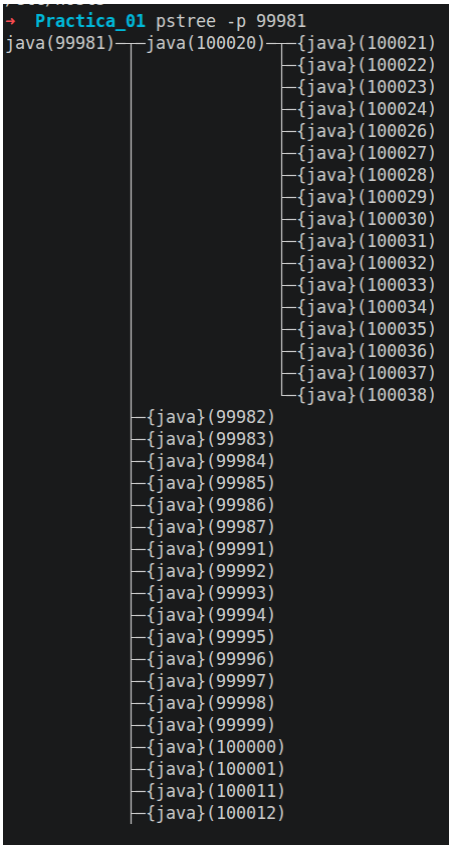


Figura 3. pstree: java(99981) supervisor → java(100020) worker; los {java} son hilos

3.3 Captura concurrente de stdout y stderr

Sin inheritIO(), los tubos del hijo van al supervisor. Dos hilos (io-stdout e io-stderr) los leen a la vez y cada línea se guarda con hora y origen. Cada línea identifica quién la ha leído (supervisor, hilo) y quién la ha escrito (worker, PID y PPID).

```
➜  supervisor git:(main) ✗ mvn -q package
java -cp target/supervisor.jar practica01.ProcessForkJoinDemo ; echo $?
java -cp target/supervisor.jar practica01.ProcessForkJoinDemo 2>/dev/null
[SUPERVISOR pid=18816 ppid=5863 thread=main] inici
[SUPERVISOR pid=18816 ppid=5863 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=18816 ppid=5863 thread=main] fill creat amb pid=18837
[SUPERVISOR pid=18816 ppid=5863 thread=io-stdout] 17:32:02.357 [stdout pid=18837] [WORKER-GAMMA pid=18837 ppid=18816 thread=main] pas 1/3
[SUPERVISOR pid=18816 ppid=5863 thread=io-stdout] 17:32:03.349 [stdout pid=18837] [WORKER-GAMMA pid=18837 ppid=18816 thread=main] pas 2/3
[SUPERVISOR pid=18816 ppid=5863 thread=io-stdout] 17:32:04.350 [stdout pid=18837] [WORKER-GAMMA pid=18837 ppid=18816 thread=main] pas 3/3
[SUPERVISOR pid=18816 ppid=5863 thread=io-stderr] 17:32:05.358 [stderr pid=18837] [WORKER-GAMMA pid=18837 ppid=18816 thread=main] acabo amb
error, codi 7
[SUPERVISOR pid=18816 ppid=5863 thread=main] fill pid=18837 ha acabat amb codi 7 (stdout=3 línies, stderr=1 línies)
0
[SUPERVISOR pid=18891 ppid=5863 thread=main] inici
[SUPERVISOR pid=18891 ppid=5863 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=18891 ppid=5863 thread=main] fill creat amb pid=18911
[SUPERVISOR pid=18891 ppid=5863 thread=io-stdout] 17:32:05.545 [stdout pid=18911] [WORKER-GAMMA pid=18911 ppid=18891 thread=main] pas 1/3
[SUPERVISOR pid=18891 ppid=5863 thread=io-stdout] 17:32:06.538 [stdout pid=18911] [WORKER-GAMMA pid=18911 ppid=18891 thread=main] pas 2/3
[SUPERVISOR pid=18891 ppid=5863 thread=io-stdout] 17:32:07.538 [stdout pid=18911] [WORKER-GAMMA pid=18911 ppid=18891 thread=main] pas 3/3
[SUPERVISOR pid=18891 ppid=5863 thread=main] fill pid=18911 ha acabat amb codi 7 (stdout=3 línies, stderr=1 línies)
➜  supervisor git:(main) ✗
```

Figura 4. stdout y stderr del worker GAMMA capturados con hora, origen, PID y PPID

3.4 Envío de datos por stdin

El supervisor escribe cinco nombres en el stdin del consumer (getOutputStream()) y lo cierra: el consumer recibe EOF y responde COUNT=5 CHARS=18.

```
➜  supervisor git:(main) ✗ java -cp target/supervisor.jar practica01.ProcessForkJoinDemo ; echo $?
[SUPERVISOR pid=25145 ppid=4861 thread=main] inici
[SUPERVISOR pid=25145 ppid=4861 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=25145 ppid=4861 thread=main] fill creat amb pid=25165
[SUPERVISOR pid=25145 ppid=4861 thread=main] stdin del fill 25165 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=25145 ppid=4861 thread=io-stdout] 10:06:21.829 [stdout pid=25165] [WORKER-GAMMA pid=25165 ppid=25145 thread=main] pas 1/3
[SUPERVISOR pid=25145 ppid=4861 thread=io-stdout] 10:06:22.818 [stdout pid=25165] [WORKER-GAMMA pid=25165 ppid=25145 thread=main] pas 2/3
[SUPERVISOR pid=25145 ppid=4861 thread=io-stdout] 10:06:23.819 [stdout pid=25165] [WORKER-GAMMA pid=25165 ppid=25145 thread=main] pas 3/3
[SUPERVISOR pid=25145 ppid=4861 thread=io-stderr] 10:06:24.822 [stderr pid=25165] [WORKER-GAMMA pid=25165 ppid=25145 thread=main] acabo amb
error, codi 7
[SUPERVISOR pid=25145 ppid=4861 thread=main] fill pid=25165 ha acabat amb codi 7 (stdout=3 línies, stderr=1 línies)
[SUPERVISOR pid=25145 ppid=4861 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, consumer]
[SUPERVISOR pid=25145 ppid=4861 thread=main] fill creat amb pid=25226
[SUPERVISOR pid=25145 ppid=4861 thread=main] stdin del fill 25226 tancat (EOF enviat, 5 línies)
[SUPERVISOR pid=25145 ppid=4861 thread=io-stderr] 10:06:25.019 [stderr pid=25226] [CONSUMER pid=25226 ppid=25145 thread=main] esperant dades
 per stdin
[SUPERVISOR pid=25145 ppid=4861 thread=io-stderr] 10:06:25.020 [stderr pid=25226] [CONSUMER pid=25226 ppid=25145 thread=main] EOF rebut
[SUPERVISOR pid=25145 ppid=4861 thread=io-stdout] 10:06:25.028 [stdout pid=25226] COUNT=5 CHARS=18
[SUPERVISOR pid=25145 ppid=4861 thread=main] fill pid=25226 ha acabat amb codi 0 (stdout=1 línies, stderr=2 línies)
[SUPERVISOR pid=25145 ppid=4861 thread=main] fi
0
```

Figura 5. El supervisor envía datos al consumer y cierra stdin (EOF)

3.5 Resultado inmutable y error explícito

```
➜  supervisor git:(main) ✗ java -cp target/supervisor.jar practica01.ProcessForkJoinDemo ; echo $?
[SUPERVISOR pid=12107 ppid=8496 thread=main] inici
[SUPERVISOR pid=12107 ppid=8496 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=12107 ppid=8496 thread=main] fill creat amb pid=12129
[SUPERVISOR pid=12107 ppid=8496 thread=main] stdin del fill 12129 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=12107 ppid=8496 thread=io-stdout] 16:39:51.150 [stdout pid=12129] [WORKER-GAMMA pid=12129 ppid=12107 thread=main] pas 1/3
[SUPERVISOR pid=12107 ppid=8496 thread=io-stdout] 16:39:52.142 [stdout pid=12129] [WORKER-GAMMA pid=12129 ppid=12107 thread=main] pas 2/3
[SUPERVISOR pid=12107 ppid=8496 thread=io-stdout] 16:39:53.142 [stdout pid=12129] [WORKER-GAMMA pid=12129 ppid=12107 thread=main] pas 3/3
[SUPERVISOR pid=12107 ppid=8496 thread=io-stderr] 16:39:54.148 [stderr pid=12129] [WORKER-GAMMA pid=12129 ppid=12107 thread=main] acabo amb
error, codi 7
[SUPERVISOR pid=12107 ppid=8496 thread=main] resultat: pid=12129 ppid=12107 codi=7 timeout=no durada=3088ms stdout=3 línies stderr=1 línies
ordre=[practica01.ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=12107 ppid=8496 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, consumer]
[SUPERVISOR pid=12107 ppid=8496 thread=main] fill creat amb pid=12204
[SUPERVISOR pid=12107 ppid=8496 thread=main] stdin del fill 12204 tancat (EOF enviat, 5 línies)
[SUPERVISOR pid=12107 ppid=8496 thread=io-stderr] 16:39:54.227 [stderr pid=12204] [CONSUMER pid=12204 ppid=12107 thread=main] esperant dades
 per stdin
[SUPERVISOR pid=12107 ppid=8496 thread=io-stderr] 16:39:54.227 [stderr pid=12204] [CONSUMER pid=12204 ppid=12107 thread=main] EOF rebut
[SUPERVISOR pid=12107 ppid=8496 thread=io-stdout] 16:39:54.231 [stdout pid=12204] COUNT=5 CHARS=18
[SUPERVISOR pid=12107 ppid=8496 thread=main] resultat: pid=12204 ppid=12107 codi=0 timeout=no durada=69ms stdout=1 línies stderr=2 línies or
dre=[practica01.ProcessForkJoinDemo, consumer]
[SUPERVISOR pid=12107 ppid=8496 thread=main] FALLADA: pid=12129 ppid=12107 codi=7 timeout=no durada=3088ms stdout=3 línies stderr=1 línies o
rdre=[practica01.ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=12107 ppid=8496 thread=main] fi: 2 processos, 1 fallits
1
```

Figura 6. Resultado de cada hijo; GAMMA (código 7) queda como FALLADA y el supervisor acaba con 1

3.6 Timeout

BETA (10 pasos) supera el límite de 5 s. El supervisor envía SIGTERM, el worker termina de forma ordenada dentro de la gracia (código 143 = 128 + 15) y ps confirma que no queda vivo.

```
→  supervisor git:(main) ✗ java -cp target/supervisor.jar practica01.ProcessForkJoinDemo ; echo $?
[SUPERVISOR pid=22526 ppid=8496 thread=main] inici
[SUPERVISOR pid=22526 ppid=8496 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, worker, ALPHA, 2, 0]
[SUPERVISOR pid=22526 ppid=8496 thread=main] fill creat amb pid=22546
[SUPERVISOR pid=22526 ppid=8496 thread=main] stdin del fill 22546 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:23.583 [stdout pid=22546] [WORKER-ALPHA pid=22546 ppid=22526 thread=main] pas 1/2
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:24.576 [stdout pid=22546] [WORKER-ALPHA pid=22546 ppid=22526 thread=main] pas 2/2
[SUPERVISOR pid=22526 ppid=8496 thread=main] resultat: pid=22546 ppid=22526 codi=0 timeout=no durada=2082ms stdout=2 línies stderr=0 línies
ordre=[practica01.ProcessForkJoinDemo, worker, ALPHA, 2, 0]
[SUPERVISOR pid=22526 ppid=8496 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, worker, BETA, 10, 0]
[SUPERVISOR pid=22526 ppid=8496 thread=main] fill creat amb pid=22596
[SUPERVISOR pid=22526 ppid=8496 thread=main] stdin del fill 22596 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:25.670 [stdout pid=22596] [WORKER-BETA pid=22596 ppid=22526 thread=main] pas 1/10
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:26.672 [stdout pid=22596] [WORKER-BETA pid=22596 ppid=22526 thread=main] pas 2/10
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:27.673 [stdout pid=22596] [WORKER-BETA pid=22596 ppid=22526 thread=main] pas 3/10
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:28.674 [stdout pid=22596] [WORKER-BETA pid=22596 ppid=22526 thread=main] pas 4/10
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:29.675 [stdout pid=22596] [WORKER-BETA pid=22596 ppid=22526 thread=main] pas 5/10
[SUPERVISOR pid=22526 ppid=8496 thread=main] TIMEOUT: el fill 22596 supera 5000ms
[SUPERVISOR pid=22526 ppid=8496 thread=main] fill 22596 aturat de forma ordenada
[SUPERVISOR pid=22526 ppid=8496 thread=main] resultat: pid=22596 ppid=22526 codi=143 timeout=sí durada=5026ms stdout=5 línies stderr=0 línie
s ordre=[practica01.ProcessForkJoinDemo, worker, BETA, 10, 0]
[SUPERVISOR pid=22526 ppid=8496 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=22526 ppid=8496 thread=main] fill creat amb pid=22662
[SUPERVISOR pid=22526 ppid=8496 thread=main] stdin del fill 22662 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:30.697 [stdout pid=22662] [WORKER-GAMMA pid=22662 ppid=22526 thread=main] pas 1/3
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:31.699 [stdout pid=22662] [WORKER-GAMMA pid=22662 ppid=22526 thread=main] pas 2/3
[SUPERVISOR pid=22526 ppid=8496 thread=io-stdout] 16:58:32.700 [stdout pid=22662] [WORKER-GAMMA pid=22662 ppid=22526 thread=main] pas 3/3
[SUPERVISOR pid=22526 ppid=8496 thread=io-stderr] 16:58:33.703 [stderr pid=22662] [WORKER-GAMMA pid=22662 ppid=22526 thread=main] acabo amb
error, codi 7
[SUPERVISOR pid=22526 ppid=8496 thread=main] resultat: pid=22662 ppid=22526 codi=7 timeout=no durada=3074ms stdout=3 línies stderr=1 línies
ordre=[practica01.ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=22526 ppid=8496 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, consumer]
[SUPERVISOR pid=22526 ppid=8496 thread=main] fill creat amb pid=22713
[SUPERVISOR pid=22526 ppid=8496 thread=main] stdin del fill 22713 tancat (EOF enviat, 5 línies)
```

Figura 7. TIMEOUT de BETA después del paso 5/10 y finalización ordenada

```
→  Practica_01 ps -eo pid,ppid,stat,cmd | grep "worker BETA"
  24130    8498 S+   grep --color=auto --exclude-dir=.bzr --exclude-dir=CVS --exclude-dir=.git --exclude-dir=.hg --exclude-dir=.svn --exclud
e-dir=.idea --exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv worker BETA
→  Practica_01 ps -eo pid,ppid,stat,cmd | grep "worker BETA"
  24478    8498 S+   grep --color=auto --exclude-dir=.bzr --exclude-dir=CVS --exclude-dir=.git --exclude-dir=.hg --exclude-dir=.svn --exclud
e-dir=.idea --exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv worker BETA
→  Practica_01
```

Figura 8. ps después del timeout: solo aparece el propio grep

3.7 Cancelación segura (sin huérfanos)

Se envía SIGTERM solo al supervisor mientras BETA trabaja. El shutdown hook registra la causa y para al hijo activo; ps no encuentra ningún worker.

```
[SUPERVISOR pid=43175 ppid=8496 thread=io-stdout] 17:30:00.865 [stdout pid=43199] [WORKER-ALPHA pid=43199 ppid=43175 thread=main] pas 2/2
[SUPERVISOR pid=43175 ppid=8496 thread=main] resultat: pid=43199 ppid=43175 codi=0 timeout=no durada=2082ms stdout=2 línies stderr=0 línies
ordre=[practica01.ProcessForkJoinDemo, worker, ALPHA, 2, 0]
[SUPERVISOR pid=43175 ppid=8496 thread=main] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar, practica01.
ProcessForkJoinDemo, worker, BETA, 10, 0]
[SUPERVISOR pid=43175 ppid=8496 thread=main] fill creat amb pid=43263
[SUPERVISOR pid=43175 ppid=8496 thread=main] stdin del fill 43263 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=43175 ppid=8496 thread=io-stdout] 17:30:01.948 [stdout pid=43263] [WORKER-BETA pid=43263 ppid=43175 thread=main] pas 1/10
[SUPERVISOR pid=43175 ppid=8496 thread=io-stdout] 17:30:02.949 [stdout pid=43263] [WORKER-BETA pid=43263 ppid=43175 thread=main] pas 2/10
[SUPERVISOR pid=43175 ppid=8496 thread=io-stdout] 17:30:03.950 [stdout pid=43263] [WORKER-BETA pid=43263 ppid=43175 thread=main] pas 3/10
[SUPERVISOR pid=43175 ppid=8496 thread=shutdown-hook] CANCEL·LACIÓ: el supervisor es tanca per un senyal (Ctrl+C o kill); aturant 1 fills ac
tius
[SUPERVISOR pid=43175 ppid=8496 thread=main] resultat: pid=43263 ppid=43175 codi=143 timeout=no durada=2212ms stdout=3 línies stderr=0 línie
s ordre=[practica01.ProcessForkJoinDemo, worker, BETA, 10, 0]
[SUPERVISOR pid=43175 ppid=8496 thread=shutdown-hook] fill 43263 aturat de forma ordenada
143
```

Figura 9. El hilo shutdown-hook para a BETA (código 143)

```
→  Practica_01 while ! pgrep -f "worker BETA" > /dev/null; do sleep 0.2; done; sleep 2; pkill -f 'ProcessForkJoinDemo$'; sleep 3; ps -eo pid
,ppid,stat,cmd | grep "worker BETA"
  43421    8498 S+   grep --color=auto --exclude-dir=.bzr --exclude-dir=CVS --exclude-dir=.git --exclude-dir=.hg --exclude-dir=.svn --exclud
e-dir=.idea --exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv worker BETA
```

Figura 10. Comprobación con ps: no queda ningún worker BETA

3.8 Pool de dos workers y pipe (ejecución completa)

ALPHA y BETA arrancan a la vez (pool-1-thread-1 y pool-1-thread-2). GAMMA espera hasta que ALPHA termina y ocupa su hilo. Nunca hay más de dos workers hijos a la vez. Después, el hilo pipe del supervisor pasa los cinco nombres del producer al consumer.

```
→  Practica_01 cd "/home/alumnat/Escriptori/Servicios y procesos/Practica_01/supervisor"
./scripts/observar-arbol.sh
Esperant que arrenqui "worker BETA"...

=== Arbre de processos amb ALPHA i BETA en marxa ===
java(54010)─┬─java(54034)
            └─java(54035)
Esperant que arrenqui "worker GAMMA"...

=== Arbre de processos quan GAMMA ocupa el lloc d'ALPHA ===
java(54010)─┬─java(54035)
            └─java(54131)

=== ps dels workers vius ===
  54035   54010 Sl+        00:03 /usr/lib/jvm/java-21-openjdk-amd64/bin/java -cp target/supervisor.jar practica01.ProcessForkJoinDemo worker
 BETA 10 0
  54131   54010 Sl+        00:01 /usr/lib/jvm/java-21-openjdk-amd64/bin/java -cp target/supervisor.jar practica01.ProcessForkJoinDemo worker
 GAMMA 3 7
```

Figura 11. Árbol de procesos: el supervisor 54010 con dos workers en cada momento

```
→  supervisor git:(main) ✗ mvn -q package
java -cp target/supervisor.jar practica01.ProcessForkJoinDemo ; echo $?
[SUPERVISOR pid=54010 ppid=8496 thread=main] inici
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-2] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar,
practica01.ProcessForkJoinDemo, worker, BETA, 10, 0]
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-1] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar,
practica01.ProcessForkJoinDemo, worker, ALPHA, 2, 0]
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-1] fill creat amb pid=54034
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-2] fill creat amb pid=54035
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-2] stdin del fill 54035 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-1] stdin del fill 54034 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:37.794 [stdout pid=54034] [WORKER-ALPHA pid=54034 ppid=54010 thread=main] pas 1/2
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:37.794 [stdout pid=54035] [WORKER-BETA pid=54035 ppid=54010 thread=main] pas 1/10
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:38.785 [stdout pid=54034] [WORKER-ALPHA pid=54034 ppid=54010 thread=main] pas 2/2
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:38.786 [stdout pid=54035] [WORKER-BETA pid=54035 ppid=54010 thread=main] pas 2/10
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:39.787 [stdout pid=54035] [WORKER-BETA pid=54035 ppid=54010 thread=main] pas 3/10
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-1] resultat: pid=54034 ppid=54010 codi=0 timeout=no durada=2071ms stdout=2 línies stder
r=0 línies ordre=[practica01.ProcessForkJoinDemo, worker, ALPHA, 2, 0]
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-1] llançant: [/usr/lib/jvm/java-21-openjdk-amd64/bin/java, -cp, target/supervisor.jar,
practica01.ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-1] fill creat amb pid=54131
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-1] stdin del fill 54131 tancat (EOF enviat, 0 línies)
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:39.862 [stdout pid=54131] [WORKER-GAMMA pid=54131 ppid=54010 thread=main] pas 1/3
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:40.788 [stdout pid=54035] [WORKER-BETA pid=54035 ppid=54010 thread=main] pas 4/10
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:40.863 [stdout pid=54131] [WORKER-GAMMA pid=54131 ppid=54010 thread=main] pas 2/3
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:41.789 [stdout pid=54035] [WORKER-BETA pid=54035 ppid=54010 thread=main] pas 5/10
[SUPERVISOR pid=54010 ppid=8496 thread=io-stdout] 17:42:41.864 [stdout pid=54131] [WORKER-GAMMA pid=54131 ppid=54010 thread=main] pas 3/3
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-2] TIMEOUT: el fill 54035 supera 5000ms
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-2] fill 54035 aturat de forma ordenada
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-2] resultat: pid=54035 ppid=54010 codi=143 timeout=sí durada=5047ms stdout=5 línies std
err=0 línies ordre=[practica01.ProcessForkJoinDemo, worker, BETA, 10, 0]
[SUPERVISOR pid=54010 ppid=8496 thread=io-stderr] 17:42:42.873 [stderr pid=54131] [WORKER-GAMMA pid=54131 ppid=54010 thread=main] acabo amb
error, codi 7
[SUPERVISOR pid=54010 ppid=8496 thread=pool-1-thread-1] resultat: pid=54131 ppid=54010 codi=7 timeout=no durada=3080ms stdout=3 línies stder
r=1 línies ordre=[practica01.ProcessForkJoinDemo, worker, GAMMA, 3, 7]
[SUPERVISOR pid=54010 ppid=8496 thread=main] pipe: producer pid=54279 --stdout--> SUPERVISOR --stdin--> consumer pid=54280
[SUPERVISOR pid=54010 ppid=8496 thread=io-stderr] 17:42:42.945 [stderr pid=54279] [PRODUCER pid=54279 ppid=54010 thread=main] generant 5 nom
s
[SUPERVISOR pid=54010 ppid=8496 thread=pipe] 17:42:42.945 [stdout pid=54279] Ana
[SUPERVISOR pid=54010 ppid=8496 thread=pipe] 17:42:42.946 [stdout pid=54279] Joan
```

Figura 12. Ejecución completa (1): pool de dos hilos, timeout de BETA, GAMMA en el hilo libre

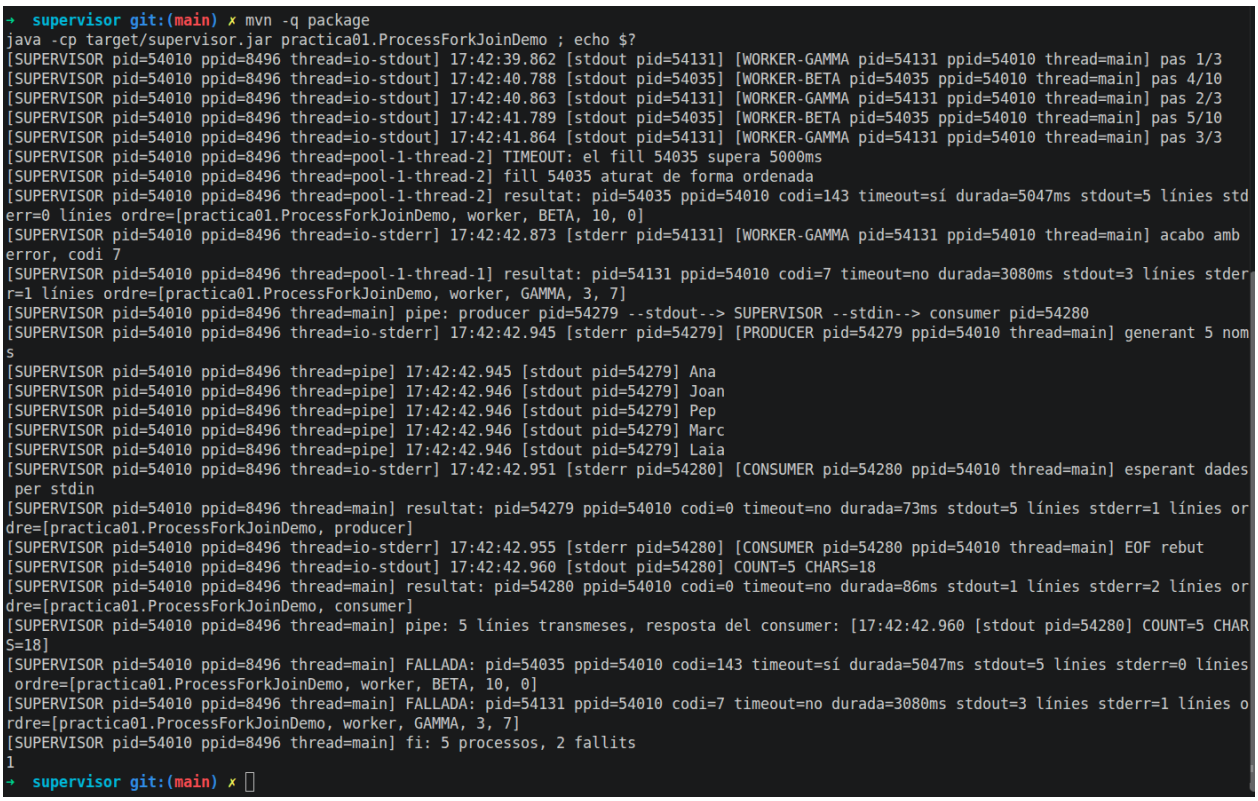


Figura 13. Ejecución completa (2): pipe producer → supervisor → consumer y resumen final

3.9 Pruebas automáticas

Nueve tests JUnit que se ejecutan con `mvn test`, sin Internet. Se han ejecutado tres veces seguidas con el mismo resultado.

Test	Qué comprueba
exito	Código 0, salida capturada, PPID = proceso de los tests, no queda vivo
ordenInexistente	Una orden que no existe produce IOException
stderrYCodigoDeError	Código 7, error en stderr con origen, resultado fallido
timeout	Límite de 1 s: timeout, código 143, no queda vivo
stdin / stdinVacio	COUNT=5 CHARS=18; con stdin vacío, COUNT=0 sin quedarse esperando
pipe	Producer y consumer hijos del supervisor, 5 líneas, COUNT=5 CHARS=18
cancelacion	La cancelación para un worker largo: código 143, no queda vivo
poolDeWorkers	ALPHA bien, BETA timeout, GAMMA código 7, ninguno vivo

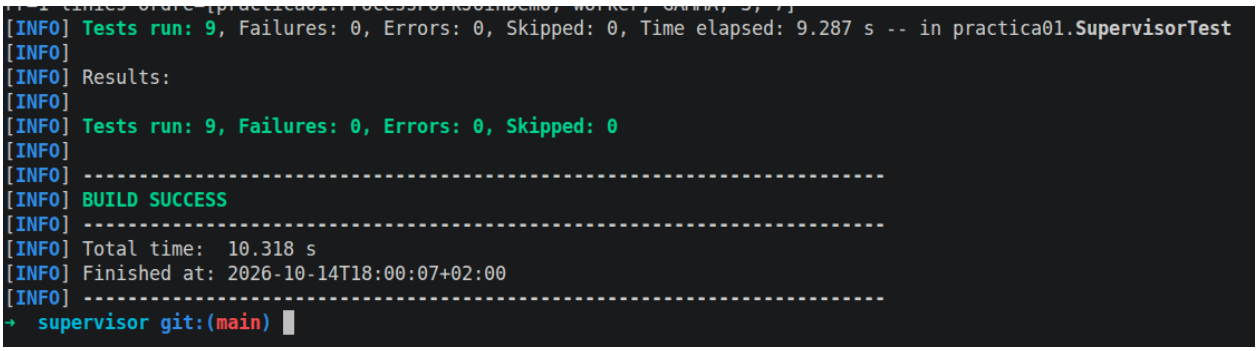


Figura 14. mvn test: 9 tests, 0 fallos

3.10 Ampliación: el supervisor como servicio systemd

Como el usuario no tiene sudo, se usa una unidad de usuario (systemctl --user): se ejecuta con la cuenta del alumno, sin privilegios, y no se habilita (no arranca sola).

```
[Unit]
Description=Supervisor multiproces - Practica 01

[Service]
Type=oneshot
WorkingDirectory=/home/alumnat/Escriptori/Servicios y procesos/Practica_01/supervisor
ExecStart=/usr/bin/java -jar "/home/alumnat/.../supervisor/target/supervisor.jar"
TimeoutStartSec=90s
KillMode=control-group
StandardOutput=journal
StandardError=journal
```

```
mkdir -p ~/.config/systemd/user
cp systemd/practica01.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user start practica01.service
systemctl --user status practica01.service --no-pager -l
journalctl --user -u practica01.service --no-pager
ps -eo pid,ppid,cmd | grep '[P]rocessForkJoinDemo' || echo "cap procés viu"
```

- El PPID del supervisor es 3546, el gestor systemd --user.
- El journal guarda las líneas reemitidas por el supervisor, con rol y PID (incluida la pipe y COUNT=5 CHARS=18).
- El servicio queda en failed con status=1 porque BETA y GAMMA fallan a propósito: es el resultado esperado. Con todos los hijos correctos sería status=0.
- Con systemctl stop a medias (código 143), el hook para a los hijos, no se lanza nada más y no queda ningún proceso vivo.
- TimeoutStartSec (90 s) es mayor que el límite de cada hijo (5 s) más la gracia (2 s).
- **¿Por qué Type=oneshot?** El supervisor hace un trabajo finito y termina; systemd espera a que acabe y registra el código. Un servicio residente (Type=simple con Restart=) tendría que quedarse vivo esperando trabajo de forma continua; con este supervisor, systemd lo vería terminar enseguida y lo reiniciaría en bucle.

```
➜  supervisor git:(main) ✗ systemctl --user daemon-reload
➜  supervisor git:(main) ✗ systemctl --user start practica01.service
Job for practica01.service failed because the control process exited with error code.
See "systemctl --user status practica01.service" and "journalctl --user -xeu practica01.service" for details.
➜  supervisor git:(main) ✗ systemctl --user status practica01.service --no-pager -l
× practica01.service - Supervisor multiproces - Practica 01
     Loaded: loaded (/home/alumnat/.config/systemd/user/practica01.service; static)
     Active: failed (Result: exit-code) since Wed 2026-10-14 18:35:34 CEST; 2s ago
    Process: 73360 ExecStart=/usr/bin/java -jar /home/alumnat/Escriptori/Servicios y procesos/Practica_01/supervisor/target/supervisor.jar (
code=exited, status=1/FAILURE)
   Main PID: 73360 (code=exited, status=1/FAILURE)
        CPU: 1.017s

d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=io-stdout] 18:35:34.038 [stdout pid=73515] COUNT=5 CHARS=18
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] resultat: pid=73515 ppid=73360 codi=0 timeout=no durada
=77ms stdout=1 línies stderr=2 línies ordre=[practica01.ProcessForkJoinDemo, consumer]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] pipe: 5 línies transmeses, resposta del consumer: [18:3
5:34.038 [stdout pid=73515] COUNT=5 CHARS=18]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] FALLADA: pid=73402 ppid=73360 codi=143 timeout=sí durad
a=5064ms stdout=5 línies stderr=0 línies ordre=[practica01.ProcessForkJoinDemo, worker, BETA, 10, 0]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] FALLADA: pid=73484 ppid=73360 codi=7 timeout=no durada=
3064ms stdout=3 línies stderr=1 línies ordre=[practica01.ProcessForkJoinDemo, worker, GAMMA, 3, 7]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] fi: 5 processos, 2 fallits
d'oct. 14 18:35:34 iesebre systemd[3546]: practica01.service: Main process exited, code=exited, status=1/FAILURE
d'oct. 14 18:35:34 iesebre systemd[3546]: practica01.service: Failed with result 'exit-code'.
d'oct. 14 18:35:34 iesebre systemd[3546]: Failed to start practica01.service - Supervisor multiproces - Practica 01.
d'oct. 14 18:35:34 iesebre systemd[3546]: practica01.service: Consumed 1.017s CPU time.
➜  supervisor git:(main) ✗ journalctl --user -u practica01.service --no-pager -n 15
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=pipe] 18:35:34.029 [stdout pid=73514] Pep
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=pipe] 18:35:34.029 [stdout pid=73514] Marc
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=pipe] 18:35:34.029 [stdout pid=73514] Laia
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] resultat: pid=73514 ppid=73360 codi=0 timeout=no durada
=66ms stdout=5 línies stderr=1 línies ordre=[practica01.ProcessForkJoinDemo, producer]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=io-stderr] 18:35:34.034 [stderr pid=73515] [CONSUMER pid=7351
5 ppid=73360 thread=main] EOF rebut
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=io-stdout] 18:35:34.038 [stdout pid=73515] COUNT=5 CHARS=18
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] resultat: pid=73515 ppid=73360 codi=0 timeout=no durada
=77ms stdout=1 línies stderr=2 línies ordre=[practica01.ProcessForkJoinDemo, consumer]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] pipe: 5 línies transmeses, resposta del consumer: [18:3
5:34.038 [stdout pid=73515] COUNT=5 CHARS=18]
```

Figura 15. systemctl start, status y journal del servicio

```
5:34.038 [stdout pid=73515] COUNT=5 CHARS=18]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] FALLADA: pid=73402 ppid=73360 codi=143 timeout=sí durad
a=5064ms stdout=5 línies stderr=0 línies ordre=[practica01.ProcessForkJoinDemo, worker, BETA, 10, 0]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] FALLADA: pid=73484 ppid=73360 codi=7 timeout=no durada=
3064ms stdout=3 línies stderr=1 línies ordre=[practica01.ProcessForkJoinDemo, worker, GAMMA, 3, 7]
d'oct. 14 18:35:34 iesebre java[73360]: [SUPERVISOR pid=73360 ppid=3546 thread=main] fi: 5 processos, 2 fallits
d'oct. 14 18:35:34 iesebre systemd[3546]: practica01.service: Main process exited, code=exited, status=1/FAILURE
d'oct. 14 18:35:34 iesebre systemd[3546]: practica01.service: Failed with result 'exit-code'.
d'oct. 14 18:35:34 iesebre systemd[3546]: Failed to start practica01.service - Supervisor multiproces - Practica 01.
d'oct. 14 18:35:34 iesebre systemd[3546]: practica01.service: Consumed 1.017s CPU time.
➜  supervisor git:(main) ✗ ps -eo pid,ppid,cmd | grep '[P]rocessForkJoinDemo' || echo "cap procés viu"
cap procés viu
➜  supervisor git:(main) ✗
```

Figura 16. Final del journal y comprobación: ningún proceso vivo

```
➜  supervisor git:(main) systemctl --user start --no-block practica01.service; sleep 3; systemctl --user stop practica01.service; journalctl
 --user -u practica01.service --no-pager -n 8; ps -eo pid,ppid,cmd | grep '[P]rocessForkJoinDemo' || echo "cap procés viu"
d'oct. 14 19:02:27 iesebre java[91320]: [SUPERVISOR pid=91320 ppid=3546 thread=pool-1-thread-1] resultat: pid=91466 ppid=91320 codi=143 time
out=no durada=884ms stdout=1 línies stderr=0 línies ordre=[practica01.ProcessForkJoinDemo, worker, GAMMA, 3, 7]
d'oct. 14 19:02:27 iesebre java[91320]: [SUPERVISOR pid=91320 ppid=3546 thread=main] el supervisor s'està tancant: no es llança [practica01.
ProcessForkJoinDemo, producer]
d'oct. 14 19:02:27 iesebre java[91320]: [SUPERVISOR pid=91320 ppid=3546 thread=main] execució cancel·lada: el supervisor s'està tancant
d'oct. 14 19:02:27 iesebre java[91320]: [SUPERVISOR pid=91320 ppid=3546 thread=shutdown-hook] fill 91364 aturat de forma ordenada
d'oct. 14 19:02:27 iesebre java[91320]: [SUPERVISOR pid=91320 ppid=3546 thread=shutdown-hook] fill 91466 aturat de forma ordenada
d'oct. 14 19:02:27 iesebre systemd[3546]: practica01.service: Main process exited, code=exited, status=143/n/a
d'oct. 14 19:02:27 iesebre systemd[3546]: practica01.service: Failed with result 'exit-code'.
d'oct. 14 19:02:27 iesebre systemd[3546]: Stopped practica01.service - Supervisor multiproces - Practica 01.
cap procés viu
```

Figura 17. systemctl stop a medias: cancelación ordenada y ningún proceso vivo

4. Mapa de procesos

Ejecución completa observada con scripts/observar-arbol.sh (pstree y ps).

Rol	PID	PPID	Estado observado	Código de retorno
Supervisor	54010	8496 (zsh)	En ejecución, padre de todos	1 (dos hijos fallidos)
Worker Alpha	54034	54010	En ejecución junto con BETA	0
Worker Beta	54035	54010	Sl+	143 (timeout: SIGTERM)
Worker Gamma	54131	54010	Sl+	7 (error)
Producer	54279	54010	Dura ~70 ms	0
Consumer	54280	54010	Dura ~90 ms	0

```
zsh (8496)
 └─ java (54010) SUPERVISOR
      ├─ java (54034) WORKER-ALPHA  ┐ a la vez
      ├─ java (54035) WORKER-BETA   ┘┐ a la vez (cuando ALPHA ha acabado)
      ├─ java (54131) WORKER-GAMMA   ┘
      ├─ java (54279) PRODUCER  ┐ unidos por la pipe del supervisor
      └─ java (54280) CONSUMER  ┘
```

Órdenes de observación y portabilidad. `ps -eo pid,ppid,stat,etime,cmd` es POSIX, pero las columnas varían entre sistemas (en macOS es command en vez de cmd). `pstree` y `ps -T` son de Linux. Ninguna funciona en Windows. Por eso la aplicación obtiene PID, PPID y descendientes con ProcessHandle (Java, portable) y no depende de ps ni de pstree para funcionar.

5. Actividades de análisis

A · ¿Procesos o hilos?

main, pool-1-thread-1/2, io-stdout, io-stderr, pipe y shutdown-hook son **hilos** del supervisor: todos aparecen con el mismo pid=54010. ALPHA, BETA, GAMMA, producer y consumer son **procesos**: cada uno tiene su PID (54034, 54035, 54131, 54279, 54280) y PPID 54010, y pstree los muestra como ramas java(PID) sin llaves (las {java} son hilos).

B · Diagnosis de errores

GAMMA acaba con el código 7. El supervisor recoge igualmente los resultados de los demás (ALPHA, BETA, producer y consumer), marca GAMMA como FALLADA por stderr y termina con código 1.

C · Bloqueo evitado

Cada tubo tiene un búfer limitado (~64 KB en Linux). Si el supervisor hiciera waitFor() antes de leer, o leyera primero un tubo y después el otro, un hijo que escribe mucho llenaría el tubo no leído y se bloquearía escribiendo; el hijo no terminaría nunca y el supervisor esperaría para siempre: interbloqueo. **Medida preventiva:** un hilo lector por tubo (LectorFlujo, hilos io-stdout e io-stderr), arrancados justo después de start(), y join() de los lectores después de waitFor() para no perder las últimas líneas.

D · Timeout

BETA tiene 10 pasos de 1 s y un límite de 5 s. Secuencia: waitFor(5000 ms) devuelve false → TIMEOUT → destroy() (SIGTERM) → el worker termina dentro de los 2 s de gracia (finalización ordenada, código 143); si no lo hubiera hecho, destroyForcibly() (SIGKILL, código 137). ps confirma que no queda vivo (figura 8).

E · Pipe

El producer escribe Ana, Joan, Pep, Marc y Laia. El consumer responde COUNT=5 CHARS=18 (3+4+3+4+4; readLine() quita los saltos de línea).

```
PRODUCER (54279)            SUPERVISOR (54010)                      CONSUMER (54280)
println("Ana") --> tubo stdout --> hilo "pipe": readLine()
                                   write("Ana") --> tubo stdin --> readLine(): lineas++, chars+=3
... Joan, Pep, Marc, Laia ...
fin del proceso -> EOF ----------> close() del stdin ----EOF---> readLine() == null
                                                                 println("COUNT=5 CHARS=18")
                   hilo io-stdout <-- tubo stdout <--------------'
```

F · Relación con Fork-Join

Fork: pool.submit() en Supervisor.ejecutarWorkers() y ProcessBuilder.start() en Supervisor.lanzar(). **Join:** waitFor() en Supervisor.esperar() y Future.get() en ejecutarWorkers(), que agrega los resultados. Los workers son **procesos** y no tareas de un ForkJoinPool porque el enunciado exige PID propio: así están aislados (memoria propia), se comunican solo por stdin/stdout/stderr y el código de retorno, y se pueden parar individualmente con señales (timeout, cancelación) sin afectar al supervisor. El precio es que crear un proceso es más costoso (~70-90 ms por JVM) que crear una tarea.

6. Criterios de aceptación

Criterio	Sí/No	Evidencia
No usa shell para concatenar entrada externa	Sí	ProcessBuilder recibe siempre una lista (Supervisor.ordenPara y lanzar); no hay sh -c. La pipe la implementa el supervisor (3.8).
No deja procesos orfes	Sí	aturar() para hijo y descendientes; shutdown hook + indicador cerrando. Figuras 8, 10 y 17 y tests timeout, cancelacion y poolDeWorkers.
Drena stdout y stderr sin interbloqueo	Sí	Un hilo lector por tubo (LectorFlujo). Figura 4 y actividad C.
Errores y timeout tienen resultado explícito	Sí	Resultado con código y timeout; FALLADA por stderr; supervisor con código 1. Figuras 6 y 7.
La batería de pruebas es reproducible	Sí	9 tests JUnit sin Internet, tres ejecuciones iguales. Figura 14.

7. Preguntas de la defensa

1. ¿Qué diferencia hay entre PID, PPID e identificador de hilo? El PID identifica un proceso; el PPID identifica al proceso que lo ha creado (su padre); el identificador de hilo (TID, columna SPID de ps -T) identifica cada hilo dentro de un proceso. Todos los hilos de un proceso comparten el mismo PID.

2. ¿Por qué ProcessBuilder tiene que recibir una lista de argumentos? Porque cada argumento llega tal cual al programa, sin una shell en medio que interprete ;, |, > o espacios. Un argumento que venga del exterior no se puede convertir en una orden nueva (inyección de órdenes).

3. ¿Por qué hay que drenar stdout y stderr concurrentemente? Porque cada tubo tiene un búfer limitado: si uno no se lee, se llena, el hijo se bloquea escribiendo y no termina, y el supervisor, que espera el otro tubo o el final del hijo, también se bloquea (interbloqueo).

4. ¿Qué aporta cerrar stdin después de escribir? Envía el EOF al hijo. Un proceso como el consumer lee hasta EOF; sin el cierre se quedaría esperando para siempre y el supervisor también, en waitFor().

5. ¿Qué diferencia hay entre finalización ordenada y forzada? La ordenada (destroy(), SIGTERM) pide al proceso que termine y le deja hacer limpieza (shutdown hooks); código 143. La forzada (destroyForcibly(), SIGKILL) la ejecuta el kernel sin que el proceso pueda reaccionar; código 137. Primero se intenta la ordenada y solo se usa la forzada si no ha terminado dentro del tiempo de gracia.

6. ¿En qué se parece esta solución a Fork-Join y en qué se diferencia? Se parece en la estructura: dividir el trabajo en tareas independientes, lanzarlas (fork) y esperar y agregar los resultados (join). Se diferencia en que cada tarea es un proceso con PID y memoria propios, y no un hilo de un ForkJoinPool: no comparten memoria, se comunican por stdin/stdout/stderr, crearlas cuesta más y se pueden parar individualmente con señales.

8. Registro de incidencias

Incidencia	Causa	Cómo se ha detectado	Solución aplicada
Error "Source option 5 is no longer supported" al compilar	Maven 3.8.7 usa por defecto maven-compiler-plugin 3.1, que ignora maven.compiler.release	Al ejecutar mvn package	Fijar maven-compiler-plugin 3.13.0 en el pom.xml
zsh: parse error al ejecutar pstree -p <PID>	Los símbolos < > del marcador se escribieron literalmente; la shell los interpreta como redirecciones	Error de la shell	Escribir solo el número
echo $? devolvía 0 en vez del código del supervisor	Se ejecutó en otra terminal: $? es el código de la última orden de esa misma shell	El valor no cuadraba con Ctrl+C	Ejecutar echo $? en la misma terminal (resultado: 130)
Worker huérfano al parar el supervisor con un worker en la cola del pool	Condición de carrera: el hook paraba los activos, pero después un hilo del pool lanzaba GAMMA, que quedaba vivo (PPID 3546)	Journal de systemd y reproducción con kill y ps	Indicador volatile cerrando: lanzar() no crea hijos durante el cierre y para los que se cuelan
Traza IllegalStateException en el journal al parar el servicio	lanzar() rechazaba crear el producer con una excepción que run() no capturaba	Captura de systemctl stop	run() la captura si el supervisor se está cerrando, escribe "execució cancel·lada" y devuelve 143

9. Instrucciones de reproducción

Requisitos: JDK 21, Maven 3 y Linux. Desde la carpeta del proyecto:

```
mvn package                                                  # compilar + tests
java -cp target/supervisor.jar practica01.ProcessForkJoinDemo ; echo $?   # demo completa
mvn test                                                     # solo los tests
./scripts/observar-arbol.sh       # en otra terminal, ANTES de lanzar la demo
```